

Technisch Ontwerp GroceryApp – Sprint 5–6 (Studentversie)

1. Inleiding

Dit document beschrijft het technische ontwerp van de GroceryApp, Sprint 5–6, studentversie. Het ontwerp is opgesteld volgens de HBO-ICT-SE richtlijnen voor technisch en functioneel ontwerp en bevat UML-diagrammen, databaseontwerp, design patterns, en sequentiediagrammen.

2. Doel van het technisch ontwerp en belanghebbenden

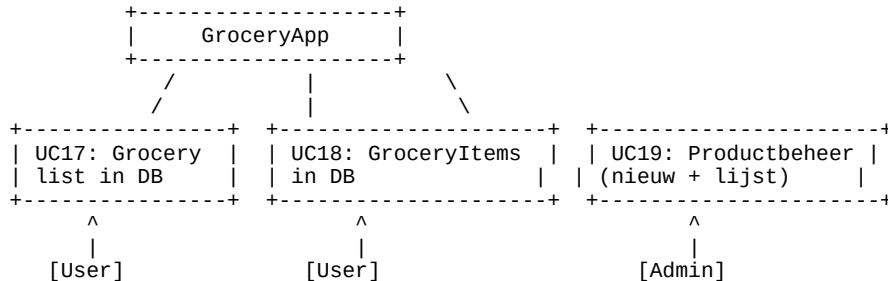
Het technisch ontwerp heeft als doel het systematisch documenteren van de structuur, functionaliteit en interacties binnen de GroceryApp, zodat het ontwikkelteam en studenten het systeem kunnen onderhouden, uitbreiden en testen.

Belanghebbenden: • Ontwikkelteam: gebruikt dit document om de code en systeemstructuur te begrijpen. • Student: aantoont competenties in analyseren en ontwerpen. • Admin/User: eindgebruikers van de GroceryApp.

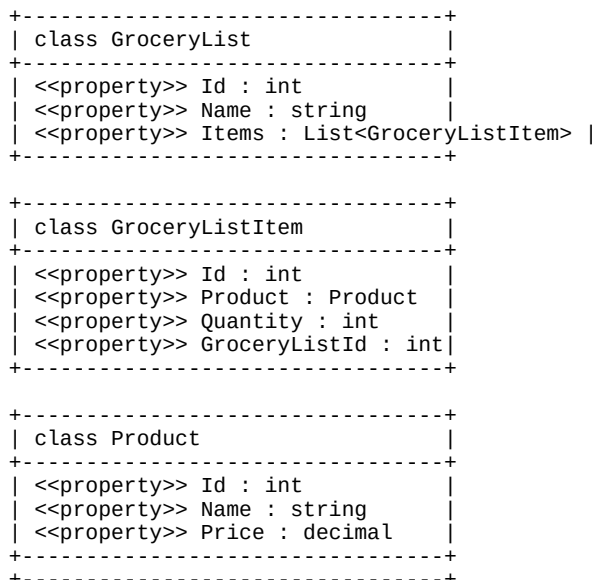
3. Ontwerp op hoofdlijnen (C4-model)

Het systeem is ontworpen volgens het C4-model op niveau 1 en 2: • Niveau 1: Contextdiagram – toont de interactie tussen de gebruiker (Admin/User) en de GroceryApp, inclusief de database. • Niveau 2: Containerdiagram – toont de applicatiecomponenten zoals ProductService, Repositories, Views en ViewModels.

4. Use Case Diagram



5. Klassendiagram (C#-stijl met <>)



```

| class GroceryListRepository      |
+-----+
| + GetAll() : List<GroceryList> |
| + GetById(id:int) : GroceryList |
| + Add(list:GroceryList) : void  |
+-----+

+-----+
| class GroceryListItemsRepository |
+-----+
| + GetAllById(id:int) : List<GroceryListItem> |
| + Add(item:GroceryListItem) : void          |
+-----+

+-----+
| class ProductRepository          |
+-----+
| + GetAll() : List<Product>      |
| + Add(product:Product) : void   |
+-----+

+-----+
| class ProductService            |
+-----+
| - _repo : ProductRepository     |
+-----+
| + GetAll() : List<Product>      |
| + Add(product:Product) : void   |
+-----+

+-----+
| class NewProductViewModel       |
+-----+
| <<property>> Name : string       |
| <<property>> Price : decimal     |
| + AddProduct() : void          |
+-----+

+-----+
| class NewProductView            |
+-----+
| <<UI>> form fields (Name, Price) |
| + OnSave() : void              |
+-----+

```

Relationships:

```

GroceryList "1" ---- "many" GroceryListItem
GroceryListItem "1" ---- "1" Product
ProductService --> ProductRepository
NewProductViewModel --> ProductService
NewProductView --> NewProductViewModel

```

6. Sequentiedigram – UC19 'Nieuw Product Aanmaken'

```
Actor(Admin)      -> NewProductView      : Voert naam en prijs in
NewProductView    -> NewProductViewModel : OnSave()
NewProductViewModel -> ProductService    : AddProduct()
ProductService     -> ProductRepository  : Add(product)
ProductRepository  -> Database            : INSERT INTO Products ...
Database           --> ProductRepository : OK
ProductRepository --> ProductService    : bevestiging
ProductService     --> NewProductViewModel : Product toegevoegd
NewProductViewModel --> NewProductView   : Toon nieuw product in lijst
NewProductView    --> Admin              : Succesmelding
```

7. Databaseontwerp

De database bevat de volgende tabellen: • Products(Id, Name, Price) • GroceryLists(Id, Name) • GroceryListItems(Id, GroceryListId, ProductId, Quantity) De repositories communiceren met deze tabellen via CRUD-operaties. Alle relaties en foreign keys zijn correct opgezet voor dataconsistentie.

8. Ontwerpdetails en design patterns

Het systeem maakt gebruik van de volgende design patterns: • Repository pattern: ProductRepository, GroceryListRepository, GroceryListItemsRepository. • MVVM pattern: Views, ViewModels en Services voor scheiding van presentatie en logica. • Singleton pattern kan toegepast worden voor centrale service-instanties indien nodig.

9. Teststrategie

Voor het systeem zijn testgevallen opgesteld voor alle use cases. Unit-tests controleren de repositories en services. UI-tests controleren de correcte werking van views en viewmodels. Database-integriteit wordt getest door CRUD-operaties op testdata.

10. Conclusie

Dit technisch ontwerp documenteert alle belangrijke componenten van de GroceryApp, volgens de HBO-ICT-SE richtlijnen. Het dient als basis voor verdere ontwikkeling, onderhoud en toetsing.